JS

WEB DEVELOPMENT

JavaScript Fundamentals

Topics: Variables & Data Types • Control Structures • Functions • Arrays & Objects

IT 201 — Introduction to Web Development

Professor: Joshua Pacer | Academic Year 2025–2026

Variables | **Functions** | **Objects** | **Arrays**

Course Overview

What we'll cover in this module

01

Role of JavaScript

Understanding JavaScript's place in web development and how it makes pages interactive.

02

Variables & Data Types

Storing and working with different kinds of information in your programs.

03

Operators

Performing calculations and comparisons to drive program logic.

04

Control Structures

Making decisions and repeating actions using conditionals and loops.

05

Functions

Organizing reusable blocks of code to avoid repetition.

06

Arrays & Objects

Storing collections of data and modeling real-world entities.

Learning Objectives

By the end of this lesson, students should be able to:

1	**Explain**	the role of JavaScript within the three-tier web development model (HTML, CSS, JS).
2	**Declare**	variables using var, let, and const and identify appropriate data types for different scenarios.
3	**Apply**	arithmetic, comparison, and logical operators to construct meaningful program expressions.
4	**Construct**	conditional statements (if/else, switch) and loops (for, while) to control program flow.
5	**Define**	and invoke functions, including arrow functions, and understand the concept of scope.
6	**Manipulate**	arrays and objects to store, retrieve, and iterate over structured data.

Role of JavaScript in Web Development

The three pillars of the modern web

HTML

Structure

Defines the skeleton and content of a page — headings, paragraphs, images, links.

CSS

Presentation

Controls layout, colors, fonts, and visual design — makes pages look beautiful.

JavaScript

Behavior

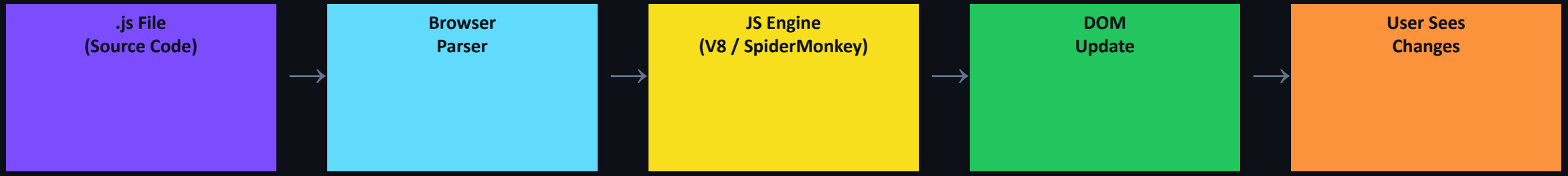
Adds interactivity, handles user events, fetches data, and updates the page dynamically.

What JavaScript Enables:

- ✔ Form validation & user input handling
- ✔ DOM manipulation (change page content dynamically)
- ✔ API calls & real-time data (AJAX / Fetch)
- ✔ Animations, games, and interactive UI

How JavaScript Works in the Browser

From source code to interactive web page



JavaScript Engine

Every browser has a built-in JS engine (Chrome uses V8, Firefox uses SpiderMonkey) that reads and executes JS code at runtime.

The DOM

Document Object Model — a tree-like representation of the HTML page that JavaScript can read and modify in real-time.

Event Loop

JS is single-threaded but handles async tasks through the event loop — a queue that processes callbacks one at a time.

Where to link JS

Use <script src="app.js"></script> before </body> or use the defer attribute to load JS without blocking the page.

Variables — Storing Information

Variables are named containers that hold values your program can use and change.

Keyword	Scope	Re-declare	Re-assign	Hoisted	Best Use
var	Function	✔ Yes	✔ Yes	✔ Yes (undefined)	Avoid in modern JS
let	Block { }	✘ No	✔ Yes	✘ No	Values that change
const	Block { }	✘ No	✘ No	✘ No	Fixed values / refs

Example:

```
// Declare variables
var oldStyle = 'avoid me';        // function-scoped, can re-declare
let score = 0;                    // block-scoped, can reassign
const MAX_LEVEL = 10;             // block-scoped, cannot reassign

score = 5;                        // ✔ Allowed with let
// MAX_LEVEL = 20;                // ✘ TypeError: Assignment to constant
```

💡 *Rule of thumb: Always use const by default. Switch to let only when you need to reassign.*

Data Types — The Building Blocks of Data

07

Every value in JavaScript belongs to a type that defines what operations you can perform on it.

Primitive Types (immutable values):

String

```
"Hello"  'World'
```

Text — wrapped in quotes. Supports template literals with backticks `Hello ${name}`

Number

```
42  3.14  -7  NaN  Infinity
```

All numbers (int & float). Special values: NaN (Not a Number), Infinity.

Boolean

```
true  /  false
```

Logical true/false. Returned by comparisons and conditions.

Undefined

```
let x;  → undefined
```

Variable declared but not yet assigned a value.

Null

```
let y = null;
```

Intentional absence of value — explicitly set by the programmer.

Symbol

```
Symbol('id')
```

Unique identifier; used as object property keys (advanced use).

Non-Primitive: Object — Arrays, functions, and objects are all of type 'object'. Use typeof to inspect: typeof 42 → 'number' | typeof 'hi' → 'string' | typeof null → 'object' (a known JS quirk!)

Operators — Working with Data

08

Operators let you perform calculations, compare values, and combine logical conditions.

⚙️ Arithmetic

+ Addition	`5 + 3 → 8`
- Subtraction	`10 - 4 → 6`
* Multiply	`6 * 7 → 42`
/ Division	`15 / 4 → 3.75`
% Modulus	`17 % 5 → 2`
** Exponent	`2 ** 8 → 256`

⚖️ Comparison

== Loose Equal	`5 == '5' → true`
=== Strict Equal	`5 === '5' → false`
!= Not Equal	`5 != 3 → true`
!== Strict Not Eq	`5 !== '5' → true`
> Greater	`8 > 3 → true`
< Less	`2 < 9 → true`
>= Greater/Eq	`5 >= 5 → true`

🔗 Logical

&& AND
`true && false → false`

|| OR
`true || false → true`

! NOT
`!true → false`

?? Nullish
`null ?? 'default'`

?: Ternary
`x>0 ? 'pos' : 'neg'`

⚠️ IMPORTANT: Always use === (strict equality) instead of == (loose equality). Loose equality does type coercion which leads to hard-to-find bugs! Example: 0 == false → true but 0 === false → false

Control Structures — Conditionals

Making decisions in code with if, else if, else, and switch statements.

if / else if / else Syntax

```
const grade = 85;

if (grade >= 90) {
    console.log('Excellent — A');
} else if (grade >= 80) {
    console.log('Good — B');      // ← This runs
} else if (grade >= 70) {
    console.log('Average — C');
} else {
    console.log('Needs improvement');
}
```

switch Statement

```
const day = 'Monday';

switch (day) {
  case 'Monday':
    console.log('Start of week!');
    break;           // ← Don't forget!
  case 'Friday':
    console.log('Almost weekend!');
    break;
  default:
    console.log('Regular day');
}
```

→ Use if/else when conditions involve ranges or complex expressions.
→ Use switch when comparing one variable against many specific values.
→ Always include break in switch cases to prevent 'fall-through' bugs.
→ The ternary operator (? :) is a shorthand for simple if/else on one line.

Control Structures — Loops

Loops repeat a block of code as long as a condition is true — essential for processing lists and automating repetition.

for Loop — Known iterations

```
// Print numbers 1 to 5
for (let i = 1; i <= 5; i++) {
    console.log(i);   // 1, 2, 3, 4, 5
}

// Loop over an array
const fruits = ['apple', 'banana', 'cherry'];
for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}
```

Anatomy of for (let i = 0; i < n; i++):

- let i = 0 ⟶ Initialize — runs once
- i < n ⟶ Condition — checked each time
- i++ ⟶ Update — runs after each iteration

while Loop — Condition-based

```
// Roll a die until you get 6
let roll = 0;
while (roll !== 6) {
    roll = Math.floor(Math.random()*6)+1;
    console.log('Rolled:', roll);
}
console.log('Got 6! Done.');
```

Modern Loop Shortcuts

```
// for...of — iterate values
for (const fruit of fruits) {
    console.log(fruit);
}
// for...in — iterate object keys
for (const key in person) { ... }
```

Functions — Reusable Blocks of Code

Functions encapsulate logic so you can reuse it, test it, and keep your code DRY (Don't Repeat Yourself).

Function Declaration

```
// Define the function
function greet(name, greeting = 'Hello') {
    return `${greeting}, ${name}!`;
}

// Call the function
console.log(greet('Maria'));          // Hello, Maria!
console.log(greet('Juan', 'Hi'));     // Hi, Juan!
```

Arrow Functions (ES6+)

```
// Traditional function
const square = function(n) {
    return n * n;
};

// Arrow function — shorter syntax
const square = (n) => n * n;

// Multi-line arrow function
const add = (a, b) => {
    const sum = a + b;
    return sum;
};
```

Scope — Where Variables Live

- Global Scope: Variables declared outside functions — accessible everywhere.
- Local/Function Scope: Variables inside a function — only accessible inside.
- Block Scope (let/const): Variables inside {} — only accessible within that block.

Functions — Parameters, Return & Callbacks

Advanced function concepts that power modern JavaScript development.

Default Parameters

```
function power(base, exp = 2) {
    return base ** exp;
}
power(3);       // 9 (exp defaults to 2)
power(3, 3);    // 27
```

Rest Parameters (...args)

```
function sum(...nums) {
    return nums.reduce((a,b) => a+b, 0);
}
sum(1, 2, 3, 4);  // 10
sum(5, 10);       // 15
```

Callback Functions

```
function doMath(a, b, operation) {
    return operation(a, b);
}
const add = (x,y) => x + y;
doMath(3, 4, add);  // 7
```

Anatomy of a Function

```
                  ┌── Parameters (inputs)
                  │        │
function multiply(a,       b) {
    return a * b;   ← Return value (output)
}
↑
Keyword (or use const/arrow)
```

Best Practices

- ✔ Use descriptive names (calculateTax, not x)
- ✔ Each function should do ONE thing
- ✔ Keep functions short (< 20 lines ideally)
- ✔ Return early to avoid deep nesting
- ✔ Document with JSDoc comments for clarity

Arrays — Ordered Collections of Data

Arrays store multiple values in a single variable, accessible by numeric index starting at 0.

Array Visualization:

'apple'	'banana'	'cherry'	'date'	'elderberry'
index [0]	index [1]	index [2]	index [3]	index [4]

Create & Access

```
const fruits = ['apple', 'banana', 'cherry'];

fruits[0];          // 'apple'   (first)
fruits[2];          // 'cherry'  (third)
fruits.length;      // 3
fruits[fruits.length-1]; // 'cherry' (last)

// Modify elements
fruits[1] = 'blueberry';
```

Common Array Methods

push()	Add to end `fruits.push('fig')`
pop()	Remove last `fruits.pop()`
unshift()	Add to start `fruits.unshift('avocado')`
shift()	Remove first `fruits.shift()`
splice()	Add/remove anywhere `fruits.splice(1,1,'grape')`
indexOf()	Find position `fruits.indexOf('apple')  // 0`
includes()	Check existence `fruits.includes('mango')  // false`
join()	Array to string `fruits.join(', ')`

Arrays — map, filter, reduce & More

ES6+ higher-order array methods let you process collections in a clean, functional style.

map()

Transform each element — returns a NEW array of same length.

```
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map(n => n * 2);

// doubled = [2, 4, 6, 8, 10]
```

filter()

Keep elements that pass a test — returns a NEW smaller array.

```
const evens = numbers.filter(n => n % 2 === 0);

// evens = [2, 4]
```

reduce()

Collapse all elements into a single value (sum, product, object, etc).

```
const total = numbers.reduce((acc, n) => acc + n, 0);

// total = 15  (0+1+2+3+4+5)
```

find()

Return the FIRST element that passes the test, or undefined.

```
const found = numbers.find(n => n > 3);

// found = 4
```

some()

Return true if AT LEAST ONE element passes the test.

```
numbers.some(n => n > 4);   // true
```

every()

Return true only if ALL elements pass the test.

```
numbers.every(n => n > 0);  // true
```

💡 *These methods don't modify the original array — they return new arrays. Chain them: numbers.filter(n=>n>2).map(n=>n*10) → [30,40,50]*

Objects — Modeling Real-World Entities

Objects group related properties and behaviors into a single unit — the foundation of data modeling in JavaScript.

Creating & Accessing Objects

```
// Object literal (most common way)
const student = {
    name:    'Maria Santos',
    age:     20,
    course:  'BSIT',
    gpa:     1.5,
    isActive: true,

    // Method — function inside object
    greet() {
        return `Hi, I'm ${this.name}!`;
    }
};

// Dot notation access
student.name;          // 'Maria Santos'
// Bracket notation access
student['course'];     // 'BSIT'
// Call method
student.greet();       // 'Hi, I'm Maria Santos!'
```

Object Operations

```
// Add new property
student.email = 'maria@school.edu';

// Update property
student.gpa = 1.25;

// Delete property
delete student.isActive;

// Check if property exists
'name' in student;  // true

// Get all keys
Object.keys(student);
// ['name','age','course','gpa','greet']

// Get all values
Object.values(student);

// Destructuring — extract to variables
const { name, course } = student;
console.log(name);    // 'Maria Santos'

// Spread — copy/merge objects
const updated = { ...student, gpa: 1.0 };
```

Arrays vs Objects — When to Use Which

Choosing the right data structure makes your code cleaner, faster, and easier to maintain.

Feature	Array [ ]	Object { }
Access by	Index number [0], [1], [2]...	Key name .name, .age...
Order	Ordered — order is preserved	Unordered (insertion order in modern JS)
Best for	Lists of similar items	Grouped properties of one thing
Example	scores = [95, 88, 76]	student = { name, age, gpa }
Iteration	for, forEach, map, filter, reduce	for...in, Object.keys, Object.entries
Length	array.length gives count	Object.keys(obj).length

Combining Both — Array of Objects (most common in real apps):

```
const students = [
    { name: 'Maria', gpa: 1.5, course: 'BSIT' },
    { name: 'Juan',  gpa: 1.8, course: 'BSCS' },
    { name: 'Ana',   gpa: 1.2, course: 'BSIT' },
];
const topStudents = students.filter(s => s.gpa <= 1.5);  // [{name:'Maria',...},{name:'Ana',...}]
```

Summary & References

Key takeaways from JavaScript Fundamentals

Role of JS

Makes web pages interactive; works with HTML & CSS in the browser DOM.

Variables

Use const by default, let when reassigning; avoid var in modern code.

Data Types

String, Number, Boolean, null, undefined, Symbol — know your types!

Operators

Use === for equality; apply arithmetic, comparison, and logical operators.

Conditionals

if/else for ranges and complex logic; switch for matching specific values.

Loops

for loops for counted iterations; while for condition-based; for...of for arrays.

Functions

Name them well, keep them small, use arrow functions for callbacks.

Arrays

Ordered lists; use .map(), .filter(), .reduce() for clean transformations.

Objects

Key-value pairs for modeling entities; use destructuring and spread.

References & Further Reading

[1] MDN Web Docs — JavaScript Reference: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[2] Eloquent JavaScript (3rd Ed.) — Marijn Haverbeke — https://eloquentjavascript.net (free online)

[3] JavaScript.info — The Modern JavaScript Tutorial — https://javascript.info

[4] You Don't Know JS (YDKJS) — Kyle Simpson — Available on GitHub: https://github.com/getify/You-Dont-Know-JS